

Nicolas Thorstensen

Diskrete Optimierung

Skriptum

Winter 2011/12

Computational Science Center
University of Vienna
A-1090 Vienna, AUSTRIA

Vorwort

Das vorliegende Skriptum basiert auf folgenden Quellen:

- Bernhard Korte and Jens Vygen, *Combinatorial Optimization*, Springer, 2000.
- William Kocaz, Donald L. Kreher *Graphs, Algorithms and Optimization*, Chapman Hall/CRC, 2000.
- Alexander Martin, *Diskrete Optimierung*, Technical University of Darmstadt, Germany, 2006.

Das einführende Kapitel wurde aus folgenden Quellen übernommen

- Otmar Scherzer and Frank Lenzen, *Optimierung*, Vorlesungsskriptum WS 2008/09, University of Innsbruck, Austria, 2009.

Nicolas Thorstensen,
Vienna, 3rd October 2011.

Inhaltsverzeichnis

1	Einleitung	1
1.1	Klassifizierung von Optimierungsproblemen	1
1.2	Beispiele	4
1.2.1	Das Zuordnungsproblem	4
1.2.2	Kapsack	5
1.2.3	Netzwerkfluß	6
1.2.4	Das Problem des Handelsreisenden	7
1.3	Komplexitätstheorie	7
2	Netzwerkprobleme	13
2.1	Grundlagen Graphen	13
2.2	Minimal Spannende Bäume	16
2.2.1	Kruskal's Algorithmus	18
2.2.2	Prim's Algorithmus	19
2.3	Kürzeste Weg Problem	19
2.3.1	Dijkstra Algorithmus	19
2.3.2	Ford-Warshall Algorithmus	21
2.4	Maximale Fluss Problem	24
2.4.1	Das Problem	24
2.4.2	Minimaler Schnitt und Maximaler Fluss	26
2.4.3	Algorithmus von Ford und Fulkerson	27
2.5	Dynamische Programmierung	32
2.5.1	Dynamisches Optimierungsproblem	32
2.5.2	Zwei Beispiele	33
2.5.3	Formale Formulierung eines diskret dynamischen Op- timierungsproblem	36
2.5.4	Dynamische Programmierung und das Bellan Optima- litätsprinzip	37

3	Lineare Programmierung	45
3.1	Lineare Programme und der Simplex Algorithmus	45
3.2	Dualität	47
3.3	Ein primal-dualer Algorithmus	50
3.3.1	Konstruktion des Algorithmus	51
4	Ganzzahlige Programmierung	57
4.1	Polyeder und ganzzahlige Polyeder	57
4.2	Unimodulare und total unimodulare Matrizen	58
4.2.1	Unimodulare Matrizen und Transformationen	58
4.2.2	Hermite Normalform	58
4.3	Formulierung als ganzzahlige programme	58

Algorithmenverzeichnis

1	Kruskal's Algorithmus	18
2	Prim's Algorithmus	20
3	Dijkstra Algorithmus	20
4	Ford-Warshall Algorithmus	22
5	Ford-Fulkerson Algorithmus	29
6	Ford-Fulkerson Markierungsalgorithmus	30
7	Dynamische Programmiermethode 1	40
8	Dynamische Programmiermethode 2	41
9	Bellman'sche dynamische Programmiermethode	42
10	Simplex Algorithmus	54
11	Primal-dualer Algorithmus	55

Kapitel 1

Einleitung

Sei $f : \Omega \rightarrow \mathbb{R}$ eine Zielfunktion die es zu minimieren gilt. Mathematische Problemstellungen der Form

$$\min_{x \in \Omega} f(x) \quad (\text{P})$$

werden *Optimierungsprobleme* genannt. In der *Optimierung* wird das Gebiet Ω *Zulässigkeitsbereich* genannt. f heißt hier *Ziel- oder Kartenfunktion*. Üblicherweise ist die Zielfunktion auf einer grösseren Menge X (zBsp : $X = \mathbb{R}^n$) definiert und die Bedingung $x \in \Omega \subset X$ ist eine Restriktion der Lösungsmenge.

Anhand der Eigenschaften von X , Ω und der Zielfunktion f lässt sich folgende grobe Einteilung von Optimierungsproblemen aufstellen. Wobei jedes dieser Optimierungsproblem mit unterschiedlichen Methoden gelöst wird.

1.1 Klassifizierung von Optimierungsproblemen

Wir treffen folgende Fallunterscheidung:

Diskrete vs stetige Optimierung

Stetige Optimierung

$X = \mathbb{R}^n$ und $\Omega \subseteq \mathbb{R}^n$.

Diskrete Optimierung

Ist der Fall wenn X oder Ω diskrete MENgen sind. Üblicherweise ist $\Omega \subset \mathbb{Z}^n$. Man spricht dann auch von *Ganzzahliger Programmierung* oder *kombinatorischen Optimierung*

Binäre Optimierung

Ist der Fall wenn der Zulässigkeitsbereich $\Omega \subset \{0, 1\}^n$. Viele Probleme der Graphentheorie lassen sich auf binäre Optimierungsprobleme zurückführen

gemischt-ganzzahlige Programmierung

In dieser Situation haben wir $X = \mathbb{R}^n$ und $\Omega \subset \mathbb{Z}^p \times \mathbb{R}^{n-p}$ mit $1 \leq p \leq n - 1$.

Unrestringierte vs restringierte Optimierung**Unrestringierte Optimierung**

Hier gilt $\Omega = \mathbb{R}^n$

Restringierte Optimierung

Fast immer lassen sich restringierte Probleme mit einem Zulässigkeitsbereich der Form $\Omega = \Omega_1 \cap \Omega_2 \cap \Omega_3$ beschreiben, wobei

$$\Omega_1 := \{x \in \mathbb{R}^n \mid c_i(x) = 0, i \in I_1\},$$

$$\Omega_2 := \{x \in \mathbb{R}^n \mid c_i(x) \leq 0, i \in I_2\},$$

$$\Omega_3 := \{x \in \mathbb{R}^n \mid x_i \in \mathbb{Z}, i \in I_3\},$$

und $I_1, I_2, I_3 \subset G = (V, E, c)$ sind paarweise disjunkte Indexmengen. Vorausgesetzt wird dabei die Stetigkeit von $c_i, i \in I_1 \cup I_2$.

Ω_1, Ω_2 und Ω_3 heißen *Gleichgewichts-, Ungleichgewichts- bzw. Ganzzahligkeitsrestriktionen*.

Stetige, restringierte Optimierungsprobleme mit $I_3 = \emptyset$ werden wie folgt unterschieden:

Klasse	f Zielfunktion	c_i Restriktionen
lineare Optimierung	linear	linear
quadratische Optimierung	quadratisch	linear
nichtlineare Optimierung mit linearen Nebenbedingungen	nichtlinear	linear
nichtlineare Optimierung mit nichtlinearen Nebenbedingungen	nichtlinear	nichtlinear

Diese Unterscheidung begründet sich vornehmlich auf die zur Lösung vornehmlich eingesetzten numerischen Verfahren.

Folgende Spezialfälle werden in der Literatur besonders betrachtet:

Lineare Optimierung:

$$f(x) = c^T x + d.$$

Quadratische Optimierung:

$f(x) = \frac{1}{2}x^T Gx + c^T x + d$ wobei $x, c \in \mathbb{R}^n$, $d \in \mathbb{R}$ und $G \in \mathbb{R}^{n \times n}$. Ohne Beschränkung der Allgemeinheit kann $d = 0$ angenommen werden.

Konvexe Optimierung:

Von konvexer Optimierung spricht man wenn die Zielfunktion konvex ist und

$$\Omega := \{x \in \mathbb{R}^n \mid c_i(x) \leq 0, i \in I\},$$

wobei alle $c_i(x)$ konvexe Funktionen sind.

Bemerkung 1.1.1. Maximierungsprobleme können wie folgt leicht auf Minimierungsprobleme zurückgeführt werden:

$$\max_{x \in \Omega} f(x) \Leftrightarrow \min_{x \in \Omega} -f(x).$$

Zusammenfassung

Im allgemeinen sind diskrete Optimierungsprobleme wesentlich schwieriger zu lösen als stetige. Ausserdem sind unrestringierte Optimierungsprobleme leichter zu lösen als restringierte. Man beachte, dass es grosse Unterschiede zwischen linearen, konvexen und nicht konvexen Problemen gibt. So lassen sich lineare Optimierungsprobleme exakt in polynomieller Zeit lösen. Konvexe Probleme lassen sich hingegen meist nur approximativ lösen. Und im Fall von nicht konvexen Problemen führen meist nicht einmal approximative Algorithmen zur Lösung. In diesem Fall erhält man dann nur lokale Lösungen.

1.2 Beispiele

Sie haben in der Vorlesung Algorithmen und Datenstrukturen bestimmt bereits von einigen der folgenden Optimierungsprobleme gehört.

1.2.1 Das Zuordnungsproblem

Gegeben:

- n Angestellte $E_1, \dots, E_n$
- n Aufgaben $T_1, \dots, T_n$
- Kosten c_{ij} , dass E_i Aufgabe T_j erledigt

Ziel

Finde eine Zuordnung von Angestellten zu Aufgaben mit minimalen Kosten

Optimierungsproblem

Sei $x_{ij} \in \{0, 1\}$, $i, j = 1, \dots, n$ definiert wie folgt

$$x_{ij} = \begin{cases} 0 & \text{wenn } E_i \text{ nicht Aufgabe } T_j \text{ zugeteilt wird} \\ 1, & \text{wenn } E_i \text{ Aufgabe } T_j \text{ zugeteilt wird.} \end{cases}$$

Dann nimmt das Zuordnungsproblem folgende Form an

$$\begin{aligned} \min \quad & \sum_{i,j=1}^n c_{ij} x_{ij} \\ \text{NB} \quad & \sum_{j=1}^n x_{ij} = 1, \quad i = 1, \dots, n, \\ & \sum_{i=1}^n x_{ij} = 1, \quad j = 1, \dots, n, \\ & x_{ij} \in \{0, 1\}, \quad i, j = 1, \dots, n. \end{aligned}$$

Weil $x_{ij} \in \{0, 1\}$ und $\sum_{j=1}^n x_{ij} = 1$ sowie $\sum_{i=1}^n x_{ij} = 1$ erledigt jeder Angestellte genau eine Aufgabe und das jede Aufgabe A_j von genau einem Angestellten erledigt wird.

Auf den ersten Blick wirkt das Problem nicht sonderlich schwierig weil es nur endlich viele Konfigurationen für $x_{i,j}$ existieren. Ein naiver Ansatz wäre alle Konfigurationen zu berechnen und jene mit minimalen Kosten zu wählen. Es stellt sich natürlich die Frage wieviele zulässige Lösungen gibt es? Sei x_{ij} als Matrix aufgefasst dann folgt aus der Nebenbedingung, dass jede Zeile und Spalte genau einen Eintrag ungleich 0 enthält. Daher haben wir n Möglichkeiten ein Element aus der ersten Zeile zu setzen. In der zweiten Zeile haben wir $(n - 1)$ Möglichkeiten. Daher ergeben sich $n!$ zulässige Lösungen. Ein naiver Algorithmus müsste alle Lösungen berechnen und mit der Zielfunktion auswerten.

Abbildung 1.1: Zuordnungsproblem

1.2.2 Knapsack

Gegeben einen Knapsack und N Objekte. Objekt x_j hat Gewicht c_j und Wert w_j für $j = 1, \dots, N$. Die Aufgabe ist den Knapsack so zu packen daß der Wert vom Inhalt maximal wird. Als Nebenbedingung haben wir ein maximales Gewicht C welches nicht überschritten werden darf. Das führt

auf folgendes Optimierungsproblem

$$\begin{aligned} & \max \sum_{i=1}^N w_j x_j \\ \text{NB} \quad & \sum_{i=1}^N c_j x_j < C \\ & x_j \in \{0, 1\}, j = 1, \dots, N. \end{aligned}$$

mit

$$x_j = \begin{cases} 1 & \text{wenn Objekt } j \text{ im Rucksack ist} \\ 0, & \text{sonst.} \end{cases}$$

Ein naiver Algorithmus überprüft alle 2^N möglichen Kombinationen.

Abbildung 1.2: *Knappsack*

1.2.3 Netzwerkfluß

Ein Energieunternehmen möchte so viel Strom wie möglich von der Produktionsstätte nach Wien transportieren. Dabei darf das Unternehmen die öffentlichen Leitungen benutzen. Da die Leitung unterschiedliche Kapazitäten haben sucht das Unternehmen jenen Pfad im Netzwerk mit maximalen Stromtransport. Dieses Problem ist bekannt als das Maximale-Flußproblem

Abbildung 1.3: *Netzwerkfluss*

1.2.4 Das Problem des Handelsreisenden

Ein Handelsreisender soll n Städte bereisen und Start und Ziel sollen die selbe Stadt sein. Es stellt sich die Frage welche Route der Handelsreisende nehmen soll, so dass die Reisekosten minimal werden aber alle Städte bereist werden. Es wird angenommen, daß die Reisekosten zwischen je zwei Städten gegeben sind.

Dieses Problem lässt sich schon graphentheoretisch beschreiben. Gegeben sei ein Graph G mit einer Knotenmenge der Mächtigkeit n (die Städte). Unter der Annahme, dass die Reisekosten unabhängig von der Richtung sind, erhalten wir einen ungerichteten Graphen. Das Optimierungsproblem

$$\min f(C) \quad \text{NB: } C \text{ sei ein hamiltonischer Kreis in } G.$$

1.3 Komplexitätstheorie

In diesem Abschnitt wollen wir die wichtigsten Begriffe und Beispiele der Komplexitätstheorie einführen. Durch den Begriff *Komplexität* wird der Aufwand zur Lösung eines Problems in Abhängigkeit der Größe der Eingangsdaten erfasst. Die Lösung wird in unserem Fall durch einen Algorithmus berechnet. Der Aufwand des Algorithmus der nötig ist, um ein Problem zu lösen

Abbildung 1.4: *Problem des Handelsreisenden*

drückt indirekt die Komplexität des Problems aus. Um Probleme untereinander vergleichbar zu machen hinsichtlich ihrer Komplexität zu vergleichen führt man Performanceklassen bzw. Komplexitätsklassen ein.

Definition 1.3.1. Es sei $\mathcal{M} = \{f | f : G = (V, E, c) \mapsto \mathbb{R}\}$ die Menge der reellwertigen Funktionen über dem Bereich der natürlichen Zahlen. Für $g \in \mathcal{M}$ sei

$$\begin{aligned}\mathcal{O}(g) &= \{f \in \mathcal{M} | \exists C, n_0 \in G = (V, E, c) : f(n) \leq cg(n), \forall n \geq n_0\} \\ \Omega(g) &= \{f \in \mathcal{M} | \exists C, n_0 \in G = (V, E, c) : f(n) \geq cg(n), \forall n \geq n_0\}\end{aligned}$$

Die Menge $\mathcal{O}(g)$ (gesprochen: 'gro Oh' von g) enthält alle Funktionen, die asymptotisch nicht schneller wachsen als die Funktion g . Die Menge $\Omega(g)$ enthält alle Funktionen, die asymptotisch mindestens so schnell wachsen wie die Funktion g . Durch obige Definitionen wird der Einfluss von Faktoren und Konstanten eliminiert.

Es sei $f(n)$ die Zahl der elementaren Schritte eines Algorithmus in Abhängigkeit der Problemgröße n . Wenn $f \in \mathcal{O}(n)$, dann ist die Laufzeit linear von oben beschränkt. Wenn $f \in \mathcal{O}(1)$, dann terminiert der Algorithmus nach einer konstanten Anzahl von Schritten und ist unabhängig von Eingangsgröße n .

Beispiel 1.3.2. $\mathcal{O}(n^{100}) \leq \mathcal{O}(2^n) \leq \mathcal{O}(n!) \quad (n! = \frac{n^n}{e^n}) \leq \mathcal{O}(n^n)$.

Definition 1.3.3. Eine Funktion $f \in \mathcal{M}$ ist polynomial, wenn es ein Polynom $g \in \mathcal{M}$ gibt, so dass $f \in \mathcal{O}(g)$ gilt.

Alle mathematischen Entscheidungsprobleme lassen sich als Sprachenprobleme formulieren. Sei B ein Algorithmus der das Entscheidungsproblem $x \in \mathcal{S}$ löst. Was ist die Worst Case Performance von B auf Problemen der Länge n ?

Definition 1.3.4. Eine Sprache $\mathcal{S}$ ist eine Menge von Zeichenketten zu einem vorgegebenen Alphabet $\mathcal{A}$. Und $\mathcal{A}^*$ ist die Menge aller möglichen Zeichenketten von $\mathcal{A}$.

Definition 1.3.5 (Entscheidungsproblem). Gegeben sein ein $x \in \mathcal{A}^*$. Das Entscheidungsproblem ist zu wissen ob $x \in \mathcal{S}$

Beispiel 1.3.6. $\mathcal{A} = \{0, \dots, 9\}$. Dann ist $\mathcal{A}^*$ die Menge aller ganzen Zahlen. Dann sei $\mathcal{S}$ die Menge aller Primzahlen. Gegeben ein x : Ist $x \in \mathcal{S}$?

Definition 1.3.7 (Worst Case Performance). R misst die Laufzeit (TIME) oder Speicherplatz (SPACE). Dann ist die Worst Case Performance

$$W(R, n, B) = \sup_{x \in \mathcal{A}^*, |x|=n} R(B(x))$$

Der Algorithmus P hat polynomiale Laufzeit, falls $W(\text{TIME}, n, B) \leq Cp(n)$ für alle n, p . p ist ein Polynom. Analog dazu hat ein Algorithmus B polynomialen Speicherbedarf, wenn $W(\text{SPACE}, n, B) \leq Cp(n)$ für alle n, p . Gibt es für ein Entscheidungsproblem einen Algorithmus mit polynomialer Laufzeit so sagt man, dass das Problem zur Komplexitätsklasse P (P-Time) gehört. Analog lässt sich die Komplexitätsklasse mit exponentieller Laufzeit definieren $W(\text{SPACE}, n, B) \leq \exp(p(n))$ (EXPSPACE und EXPTIME).

Eine weitere Möglichkeit der Aufwandsbewertung für die Lösung eines Problems besteht darin zu untersuchen wie groß der algorithmische Aufwand ist, die Lösung eines Entscheidungsproblems (mit Begründung) auf Richtigkeit zu testen. Man wählt wieder einen Algorithmus $\mathcal{A}$ und wendet ihn auf eine Instanz des Entscheidungsproblems und eine 'eratene' Lösung ($x \in L$) an.

$$\begin{aligned} \mathcal{N}(\mathcal{A}, x, \mathcal{L}_x, \mathcal{R}) \dots & \text{Aufwand in Ressourcen } \mathcal{R} \text{ für die Überprüfung der} \\ & \text{Lösung } \mathcal{L}_x \text{ für } x \in L \\ \sup_{x \in \mathcal{A}^*, |x|=n} \mathcal{N}(\mathcal{A}, x, \mathcal{R}) \dots & \text{Worst case, den Algorithmus zur Überprüfung der} \\ & \text{Lösung für Instanzen der Größe } n \text{ benötigt} \end{aligned}$$

Wir sagen ein Problem gehört zur Problemklasse $NP(\text{TIME})$ falls ein Algorithmus $\mathcal{A}$ existiert und ein Polynom p , sodaß $\mathcal{N}(\Omega, n, \text{TIME}) \leq p(n)$ $n \in G = (V, E, c)$.

NPSPACE

Falls ein Algorithmus $\mathcal{A}$ und ein Polynom p existieren mit

$$\mathcal{N}(\mathcal{A}, x, \text{Space}) \leq p(n) \quad \forall n \in G = (V, E, c).$$

NEXPTIME

Falls ein Algorithmus $\mathcal{A}$ und ein Polynom p existieren mit

$$\mathcal{N}(\mathcal{A}, x, \text{TIME}) \leq \exp(p(n)) \quad \forall n \in G = (V, E, c).$$

NEXPSPACE

Falls ein Algorithmus $\mathcal{A}$ und ein Polynom p existieren mit

$$\mathcal{N}(\mathcal{A}, x, \text{SPACE}) \leq \exp(p(n)) \quad \forall n \in G = (V, E, c)$$

Statt $\sup_{x \in \mathcal{A}^*, |x|=n} \mathcal{N}(\mathcal{A}, x, \mathcal{R})$ kann man auch $\mathbb{E}(\mathcal{N}(\mathcal{A}, x, \mathcal{R}), |x| = n)$ betrachten, unter der Annahme einer Wahrscheinlichkeitsverteilung für die $x \in \mathcal{A}^*$. Das führt zu Komplexitätsklassen der erwarteten Komplexität, daß heißt Komplexitätsklassen hängen von Wahrscheinlichkeitsverteilungen ab. Nun wollen wir beschreiben wie die Komplexitätsklassen zusammenhängen. Aus der einfachen Überlegung, daß jeder Speicherzugriff Zeit benötigt haben wir folgende Einbettungen

$$\begin{aligned} P &\subset PSPACE \\ NP &\subset NPSPACE \\ EXPTIME &\subset EXPSPACE \\ NEXPTIME &\subset NEXPSPACE \end{aligned}$$

ausserdem, wenn es ein Polynom $p(n) \leq \exp(q(n))$ für geeignetes q (odBA sogar für linear q) existiert, dann gilt

$$\begin{aligned} P &\subset EXPTIME \\ NP &\subset NEXPTIME \\ PSPACE &\subset EXPSPACE \\ NPSPACE &\subset NEXPSPACE \end{aligned}$$

wobei das NP für nicht deterministisch polynomial steht. Das Überprüfen der Lösung ist im Finden inkludiert also

$$\begin{aligned} P &\subset NP \\ EXPTIME &\subset NEXPTIME \\ EXPSPACE &\subset NEXPSPACE \end{aligned}$$

Es gelten folgende mathematischen Sätze in der Komplexitätstheorie

$$\begin{aligned} P &\neq EXPTIME \\ PSPACE &\neq EXPSPACE \\ NP &\neq NEXPSPACE \\ NPSPACE &\neq NEXPSPACE \end{aligned}$$

Wie untersucht man die Zusammenhänge zwischen Komplexitätsklassen? Dafür betrachten wir zwei Problem P_1 und P_2 ; Sei $P_1 \in P$ Falls ein Algorithmus $\mathcal{A}_{12}$ existiert mit dessen Hilfe eine Instanz von P_2 in eine Instanz von P_1 überführen kann und die Lösung von P_1 in eine Lösung von P_2 übersetzt werden kann und $\mathcal{A}_{12}$ polynomialen Aufwand in $TIME$ hat, dann ist auch $P_2 \in P$. Wir können P_1 und P_2 als "polynomial" äquivalent betrachten. Man spricht auch oft von einer Reduktion.

Wir betrachten eine Komplexitätsklasse $\mathcal{K}$. Ein Problem Q_1 heißt $\mathcal{K}$ -hart, wenn es mindestens so schwierig ist wie jedes Problem in $\mathcal{K}$. Das heißt, wenn es für jedes Problem $Q \in \mathcal{K}$ einen zu $\mathcal{K}$ Übersetzungsalgorithmus gibt, um Q_2 in Q_1 zu übersetzen. Ein Problem heißt $\mathcal{K}$ -vollständig wenn $Q_1 \in \mathcal{K}$ und $Q_1 \mathcal{K}$ -hart ist.

Theoretisch reichen einzelne NP -vollständige Probleme um die Frage $P = NP$ zu entscheiden. Wir könnten zBsp anhand von Minesweeper, Tetris oder Sudoku die Frage $P = NP$ untersuchen.

Komplexitätszoo von NP-vollständigen Problemen

3-SAT

Gegeben eine Menge $c_1, \dots, c_m$ von Ausdrücken mit n logischen Variablen $x_1, \dots, x_n$, sodaß jeder Ausdruck von höchstens drei Variablen abhängt und folgende Form hat $y_i \vee y_l \vee y_k$ wobei y_i entweder x_i oder $\neg x_i$.

$n = 4, m = 3$ eine mögliche Instanz ist

$$C_1 = x_1 \neg x_2 \neg x_3$$

$$C_2 = x_2 \neg x_3 \neg x_4$$

$$C_3 = \neg x_1 \neg x_3 \neg x_4.$$

Entscheidungsfrage: Kann jeder Variable ein Wert zugewiesen werden, sodaß alle C_i wahr sind? Dieses Problem ist das Erste von dem bewiesen wurde, daßes NP -vollständig ist.

3-Farben Färbung

Kann man gegebene Landkarte mit 3 Farben färben, sodaß benachbarte Knoten nicht die selbe Farbe teilen?

Monochromatische Dreiecke

Gegeben sei ein ungerichteter Graph G . Kann man die Kanten E in zwei disjunkte Teilmengen zerlegen, sodaß keiner der Graphen G_1 und G_2 ein Dreieck enthält?

Clique

Gegeben sei ein ungerichteter Graph G mit n Knoten und ein $k \leq n$. Enthält G eine k -Clique, das heißt eine k elementige Teilmenge W von Knoten, sodaß $G[W]$ ein Graph ist indem je zwei Knoten miteinander verbunden sind?

Multiprozessor Scheduling

Gegeben ist eine Menge von Aufgaben $\{A_i\}$ und für jedes $t \in T$ eine positive Laufzeit mit $length(t) \in G = (V, E, c)$ und einer deadline. Gibt es eine Planung für zwei Prozessoren so dass alle Aufgaben bis zur deadline erledigt sind? ...

Entscheidungsbäume

Gegeben sei eine logische Funktion f in n Variablen, die beschrieben ist durch die Menge ihrer Wahr-Punkte, das heißt durch

$$A = \{(a_1, \dots, a_n) | f(a_1, \dots, a_n) = 1\}.$$

Gibt es einen Entscheidungsbaum für f , dessen durchschnittliche Pfadlänge höchstens K für gegebenes K ?

Aufspannender Baum mit maximales Blattzahl

Gegeben sei ein ungerichteter Graph G mit n Knoten und ein $k \leq n$. Gibt es einen aufspannenden Baum von G , für den mindestens k Knoten vom Grad 1, das heißt Blätter sind?

Längste Kreis

Gegeben sei ein ungerichteter Graph G mit n Knoten und ein $k \leq n$. Enthält G einen einfachen Kreis der mindestens k Knoten enthält?

Handelsreisende

Haben wir bereits im ersten Abschnitt bereits eingeführt. Also, gibt es einen Hamiltonschen Kreis mit Kantengewichtssumme $\leq C$?

Netzwekoptimierungsprobleme

Fast alle aber nicht Single commodity netzwerkfluß.

Allgemein kann man sagen, daß fast alle kombinatorischen Optimierungsprobleme im Worst-Case NP -vollständig sind. Allerdings ist nicht gesagt, daß ein Großteil der Probleme in polynomieller Zeit lösbar sind.

Kapitel 2

Netzwerkprobleme

In dieser Vorlesung werden wir uns auf diskrete Optimierungsprobleme beschränken, die sich als Netzwerkprobleme bzw. als Graphenprobleme modellieren lassen. Dafür werden wir ersten Abschnitt die wichtigsten Objekte der Graphentheorie die für uns von Relevanz sind einführen. Dieser Abschnitt orientiert sich im wesentlichen am 2.ten Kapitel vom Buch von Bernhard Korte und Jens Vygen Diese Definitionen werden uns im Laufe der Vorlesung immer wieder begegnen. Nach den einführenden Definitionen im Abschnitt 2.1, werden wir in Abschnitt 2.3 das Problem den kürzesten Weg zwischen zwei vorgegebenen Knoten zu berechnen. In Abschnitt 2.4 werden wir uns mit Netzwerkflüssen und der algorithmischen Berechnung beschäftigen. Im letzten Teil 2.5.1 führen wir die dynamische Programmierung ein, weil viele kombinatorische Probleme als dynamisches Optimierungsproblem formulieren lassen.

2.1 Grundlagen Graphen

Ein Graph ist ein Tupel $G = (V, E)$ mit $V \neq \{\}$ eine endliche Menge von Knoten und Kanten $E \subset V \times V$. Sei $e = (v, w) \in E$ eine Kante mit $v, w \in V$. Dann gilt

- i) $e = (v, w)$ verbindet v und w . Man sagt v und w sind benachbarte Knoten.
- ii) v ist ein Nachbar von w und w ist ein Nachbar von v . Die Menge aller Nachbarknoten von v ist $N(v)$.
- iii) v und w sind die Endknoten von e .
- iv) v und w sind die inzidenten Knoten von e .

Definition 2.1.1 (Gerichtete, ungerichtete Graphen, unterliegende ungerichtete Graphen)

- i) Ein Graph $G = (V, E)$ heisst ungerichtet, wenn alle Kanten $e \in E$ ungerichtet sind, d.h alle Paare $(v, w) \in E$ sind nicht geordnet.
- ii) Ein Graph $G = (V, E)$ heisst Digraph, wenn alle Kanten $e \in E$ gerichtet sind, also alle Paare $(v, w) \in E$ geordnet sind.
- iii) Der induzierte ungerichtete Graph eines Digraphen ist der ungerichtete Graph G' mit $(v, w) \in E \Leftrightarrow (v, w) \in E'$. Der ungerichtete Graph hat die gleiche Anzahl Knoten und gerichtete Kanten werden durch ungerichtete ersetzt
- iv) v und w sind die inzidenten Knoten von e .

In einem gerichteten Graphen ist ein Knoten ohne Vorgänger eine Quelle Q und ein Knoten ohne Nachfolger eine Senke S . Die Menge aller Nachfolger von einem Knoten v ist $S(v)$ und die Menge aller Vorgänger ist $P(v)$.

Beispiel 2.1.2.

$N(1) = \{S, 2, 3, 4\}$; $E = \{(S, 1), (S, 2), \dots\}$; $S(2) = \{1, 3\}$, $P(4) = \{1\}$.

Abbildung 2.1: Beispiele zu Graphen und Digraphen

Definition 2.1.3 (vollständiger Graph).

Ein Graph $G = (V, E)$ ist vollständig wenn für alle Knoten $v, w \in V$ gilt $(v, w) \in E$. Also, je zwei Knoten im Graphen sind miteinander verbunden.

Definition 2.1.4.

Sei $G = (V, E)$ ein Digraph und $e = (v, w) \in E$ eine Kante. Dann sagen wir

- i) e verlässt v und trifft w .
- ii) v ist das Ende und w der Kopf von e .
- iii) v ist der Vorgänger von w . Die Menge aller Vorgänger von w ist $P(w)$ und $\delta^-(w) = |P(w)|$ ist der Eingangsgrad von w .
- iv) w ist der Nachfolger von v . Die Menge aller Nachfolger von v ist $S(v)$ und $\delta^+(v) = |S(v)|$ ist der Ausgangsgrad von v .

Definition 2.1.5 (Wege, Pfade).

Sei $G = (V, E)$ ein Graph.

- i) Eine Folge von Knoten und Kanten $W = (v_1, e_1, \dots, v_k, e_k, v_{k+1})$ mit $k \geq 0$ und $v_i \in V, e_j \in E$ ist ein Weg, wenn $e_i \neq e_j \forall 1 \leq i, j \leq k$
- ii) Ein Weg ist ein Pfad wenn $v_i \neq v_j \forall 1 \leq i, j \leq k + 1$. Die Länge eines Pfades ist die Anzahl seiner Kanten.
- iii) Ein Weg ist geschlossen, wenn $v_1 = v_{k+1}$. Dann ist W ein Kreis.
- iv) Ein Knoten w heisst erreichbar, wenn ein $v - w$ Pfad existiert.

Beispiel 2.1.6.

Finde einen Weg zwischen Q und S .

Definition 2.1.7 (Zusammenhängender Graph).

- i) Ein Graph $G = (V, E)$ heisst zusammenhängend, wenn ein $v - w$ Pfad existiert für alle $v, w \in V$.
- ii) Ein Digraph $G = (V, E)$ heisst zusammenhängend, wenn der induzierte ungerichtete Graph zusammenhängend ist.

Definition 2.1.8 (Gewichteter Graph, Netzwerk).

Abbildung 2.2: *Weg und Kreis*

- i) Ein gewichteter Graph oder Netzwerk N ist ein Tripel $G = (V, E, c)$. (V, E) ist ein Graph und $c : V \mapsto \mathbb{R}$ heisst Gewichtsfunction mit $e = (i, j) \in E \mapsto c(e) = c_{ij}$.
- ii) Das Netzwerk heisst ungerichtet, wenn der Graph ungerichtet ist.
- iii) Das Netzwerk heisst gerichtet, wenn der Graph gerichtet ist.

Beispiel 2.1.9 (Ein Netzwerk).

$G = (V, E, c)$ mit Kostenfunktion c

2.2 Minimal Spannende Bäume

Anwendungsbeispiele:

- Computernetwerke
- Telefon und elektrische Netzwerke

Wir beginnen den Abschnitt mit der Wiederholung von Bäume in der Graphentheorie.

Definition 2.2.1 (Baum, aufspannender Baum).

Abbildung 2.3: *Netzwerk*

- i) $G' = (V', E')$ ist ein Teilgraph von $G = (V, E)$, wenn $V' \subseteq V$ und $E' \subseteq E$. Ein Teilgraph G' heisst aufspannend, wenn $V' = V$.
- ii) Ein zusammenhängend, kreisfreier und ungerichteter Graph heisst Baum.
- iii) Ein aufspannender Baum eines ungerichteten Graphen ist ein aufspannender Teilgraph mit der Eigenschaft ein Baum zu sein.

Beispiel 2.2.2 (Baum, aufspannender Baum).

Siehe Abbildung 2.4.

Theorem 2.2.3.

Sei $G = (V, E)$ ein ungerichteter Graph mit n Knoten. Die folgenden Aussagen sind äquivalent:

- i) G ist ein Baum.*
- ii) G hat $(n - 1)$ Kanten und enthält keine Kreise.*
- iii) G ist ein maximal kreisfreier Graph (hinzufügen einer Kante erzeugt einen Kreis.).*
- iv) Für zwei Knoten v, w existiert genau ein Weg.*

Beweis.

Übung.

Abbildung 2.4: *Baum, aufspannender Baum*

2.2.1 Kruskal's Algorithmus

Das Problem:

Sei $G = (V, E)$ ein ungerichteter Graph mit Knoten $V = \{v_1, \dots, v_n\}$ und Kanten E mit $c : E \mapsto \mathbb{R}$. Finde einen aufspannenden Baum $T(V, E')$ in G mit $E' \subset E$, sodass $\sum_{e \in E'} c(e)$ minimal wird.

Algorithmus : Kruskal

Daten : $G = (V, E)$

Ergebnis : A set $E' \subset E$;

Initialisierung : Set $F := \emptyset$;

Ordne die Elemente von E in aufsteigender Reihenfolge
 $w(e_1) \leq w(e_2) \leq \dots \leq w(e_n)$;

für alle $i = 1, \dots, n$ **tue**

wenn $(V, E' \cup \{e_i\})$ *enthält keinen Kreis* **dann**

$E' = E' \cup \{e_i\}$ und $T = (V, E')$

Ende

Ende

Algorithmus 1: Kruskal Algorithmus

Beispiel 2.2.4 (Kruskal' Algorithmus).

Siehe Abbildung 2.5.

Abbildung 2.5: *Baum, aufspannender Baum*

Theorem 2.2.5. *Kruskal's Algorithmus berechnet einen spannednen Baum mit minimalen Kosten, wenn $G = (V, E)$ einen spannenden Baum besitzt.*

Beweis. Korte and Vygen, Theorem 6.3, p. 129.

Die Komplexität von Kruskal's Algorithmus hängt von der Komplexität des Sortieralgorithmus ab und der Überprüfung, daß kein Kreis erzeugt wird. Sortieren läßt sich eine beliebige Liste mit Merge-Sort in $O(n \log n)$ und

2.2.2 Prim's Algorithmus

Siehe algorithmus 2.

2.3 Kürzeste Weg Problem

2.3.1 Dijkstra Algorithmus

Ausgabe :

- Kürzesten Weg von $Q = 1$ zu allen $i \in V$ und die dazugehörige Länge.

Algorithmus : Prim

Daten : $G = (V, E, c)$

Ergebnis : A set $E' \subset E$;

Initialisierung : Set $F := \emptyset$;

Ordne die Elemente von E in aufsteigender Reihenfolge
 $w(e_1) \geq w(e_2) \geq \dots \geq w(e_n)$;

für alle $i = 1, \dots, n$ **tue**

wenn $F \cup \{e_i\} \in \mathcal{F}$ **dann**

$F \leftarrow F \cup \{e_i\}$;

Ende

Ende

Algorithmus 2: Prim's Algorithmus

Algorithmus : Dijkstra Algorithmus

Daten : $G = (V, E, c)$, Q und S

Initialisierung : Setze $W := \{1\}$; $d(1) = 0$; $d(i) = \infty$; $\forall i \in V \setminus \{1\}$

für alle $i \in V \setminus \{1\}$ **tue**

wenn $(1, i) \in E$ **dann**

 Setze $d(i) = c_{1i}$ und $p(i) = 1$

Ende

Ende

solange $W \neq V$ **tue**

 Finde $k \in V \setminus W$ mit $d(k) = \min\{d(i) | i \in V \setminus W\}$.

 Setze $W = W \cup k$.

für alle $i \in V \setminus W$ mit $(k, i) \in E$ **tue**

wenn $d(i) > d(k) + c_{ki}$ **dann**

 Setze $d(i) = d(k) + c_{ki}$ und $p(i) = k$

Ende

Ende

Ende

Algorithmus 3: Dijkstra Algorithmus

- $i \in V$, $d(i)$ ist die Länge vom kürzesten $Q - i$ -Weg. $p(i)$ ist der Vorgänger von i am kürzesten Weg.
- Wenn i nicht erreichbar von Q , dann $d(i) = \infty$ und $p(i)$ undefiniert.

Komplexität :

- While Schleife $n - 1$ Iterationen.
- n Schritte um das Minimum $d(k)$ zu finden.
- Daher Dijkstra $\in O(n^2)$ n ist die Anzahl Knoten.

Bemerkung 2.3.1.

- Dijkstra konvergiert nur dann sicher wenn $c_{ij} \geq 0$

Beispiel 2.3.2 (Bsp Dijkstra).

Wenden sie den Algorithmus von Dijkstra auf folgendes Netzwerk N an.

Abbildung 2.6: *Beispiel mit Dijkstra Algorithmus*

2.3.2 Ford-Warshall Algorithmus

Dieser Algorithmus funktioniert auch mit negativen Kantengewichten. Allerdings darf das Netzwerk keine Kreise negativer Länge enthalten. Enthält das Netzwerk einen Kreis negativer Länge, so wird dieser gefunden. Ansonsten berechnet der Algorithmus nicht nur einen $Q - S$ kürzesten Weg, sondern alle kürzesten Wege für alle Paare (i, j) mit $i, j \in V$.

Die Ausgabe des Ford-Warshall Algorithmus sind zwei Matrizen:

Algorithmus : Ford-Warshall Algorithmus

Daten : $G = (V, E, c)$

Ergebnis : A set $E' \subset E$;

Initialisierung : Set $F := \emptyset$;

Ordne die Elemente von E in aufsteigender Reihenfolge

$w(e_1) \geq w(e_2) \geq \dots \geq w(e_n)$;

```

für alle  $j = 1, \dots, n$  tue
  |
  | für alle  $i = 1, \dots, n$  mit  $j \neq i$  tue
  | |
  | | für alle  $k = 1, \dots, n$  mit  $j \neq k$  tue
  | | |
  | | | wenn  $d_{ik} > d_{ij} + d_{jk}$  dann
  | | | |  $d_{ik} = d_{ij} + d_{jk}$  und  $p_{ik} = p_{jk}$ 
  | | | Ende
  | | Ende
  | Ende
Ende

```

Algorithmus 4: Ford-Warshall Algorithmus

- Die Distanzmatrix

$$D = \begin{pmatrix} d_{11} & d_{12} & \cdots & d_{1n} \\ d_{21} & \ddots & \ddots & d_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ d_{n1} & d_{n2} & \cdots & d_{nn} \end{pmatrix},$$

mit d_{ij} ist die Länge des kürzesten Weges von i nach j .

- Vorgänger Matrix

$$P = \begin{pmatrix} p_{11} & p_{12} & \cdots & p_{1n} \\ p_{21} & \ddots & \ddots & p_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ p_{n1} & p_{n2} & \cdots & p_{nn} \end{pmatrix},$$

wobei (p_{ij}, j) die letzte Kante des kürzesten Weges von i nach j . Also p_{ij} ist der Vorgänger von j im kürzesten Weg von i nach j (falls einer existiert).

Bemerkung 2.3.3 (Erkennung von Kreisen). Ein Diagonalelemente d_{ii} ist negativ genau dann wenn das Netzwerk Kreis negativer Länge enthält. Wenn eines der $d_{ii} < 0$, dann terminiert der Floyd-Warshall Algorithmus.

Beispiel 2.3.4.

Wenden sie den Algorithmus von Floyd-Warshall auf folgendes Netzwerk N an.

Kürzeste Weg zwischen 1 und 3.

Abbildung 2.7: *Beispiel mit Floyd-Warshall Algorithmus*

- p_{13} , dann Vorgänger ist 2
- p_{12} , dann Vorgänger ist 4
- p_{14} , dann Vorgänger ist 1

Dann ist der kürzeste Weg $W = (1, 2, 3, 4)$.

Die Annahme die hinter der Zurückverfolgung steckt ist, daß Teilwege von kürzesten Wegen auch kürzeste Wege sind.

Theorem 2.3.5.

Der Algorithmus von Floyd-Warshall berechnet für alle $i, j \in V$ die Länge des kürzesten Weges von i nach j , wenn keine Kreise negativer Länge enthalten sind in N .

Beweis.

Übung. Der Beweis wird induktiv geführt.

2.4 Maximale Fluss Problem

In diesem Abschnitt beschäftigen wir uns mit einer speziellen Klasse von Netzwerkproblemen: **dem maximalen Fluss Problem** (Maximum flow problem) und die algorithmische Umsetzung anhand des Algorithmus von Ford und Fulkerson. Dieser spezielle Algorithmus hat polynomielle Laufzeit in der Anzahl der Knoten. Wir haben bereits ein Problem dieser Klasse in der Einleitung kennengelernt.

2.4.1 Das Problem

Sei $N = (V, E, u)$ mit Knotenmenge V und Kanten E mit $|E| = m$. Weiters sei $u : E \mapsto \mathbb{R}_+$ mit $(i, j) \in E \mapsto u(i, j) = u_{ij}$ die maximale Kapazität einer Kante. Ausserdem, sei $x : E \mapsto \mathbb{R}$ mit $(i, j) \in E \mapsto x(i, j) = x_{ij}$ die Saturation, also die Menge an Waren die über die Kante transportiert werden. Die Quelle Q und Senke S sind ausserordentliche Knoten. Die verbleibenden Knoten sind die *inneren* Knoten.

Bemerkung 2.4.1.

- Q hat keine Vorgänger und S hat keinen Nachfolger.
- Alle s_{ij} erfüllen folgende Kapazitätsbedingung

$$0 \leq s_{ij} \leq u_{ij}, \quad 0 \leq u_{ij}, \quad \forall (i, j) \in E$$

- Erhaltungsgleichung: Abgesehen von der Quelle und der Senke muss in jeden Knoten genau so viel hinein- wie herausfließen

$$\underbrace{\sum_{k \in S(j)} x_{jk}}_{\text{Abfluss vom Knoten } j} - \underbrace{\sum_{k \in P(j)} x_{k,j}}_{\text{Zufluss in den Knoten } j} = 0$$

Bemerkung 2.4.2.

Es treten in der Praxis Netzwerkprobleme mit mehreren Quellen und Senken auf. Diese lassen sich transformieren in das kononische Problem mit einer Quelle und einer Senke. Diese Transformation erreicht man üblicherweise indem zBsp. ein künstlicher Knoten eingeführt wird. Diese 'Super Quelle' wird mit Kanten von unendlich grosser Kapazität mit den vielen ursprünglichen Quellen verbunden. Analog wird vorgegangen, wenn mehrere Senke im Problem auftreten. Somit sind dann wieder alle Annahmen erfüllt (eben Annahme i)).

Beispiel 2.4.3.

Zurückführen auf Standardform (Siehe Abbildung 2.8).

Abbildung 2.8: *Mehrere Quellen und Senken*

Definition 2.4.4 (Fluss).

- Ein Fluss $x : E \mapsto \mathbb{R}$ ist
- Ein Fluss x heißt zulässig, wenn die Kapazitätsbedingung erfüllt ist.
- Der Wert von einem Fluss x ist

$$w = \sum_{(Q,j) \in E} x_{Qj} = \sum_{(i,S) \in E} x_{iS}$$

- Ein möglicher Fluss mit maximalem Wert heißt **maximaler Fluss**.

Wir sind nun in der Lage das Maximale Fluss Problem zu formulieren.

Maximale Fluss Problem:

Finde einen maximalen Fluß x im Netzwerk N .

Das Problem ist lösbar, wenn x alle Bedingungen erfüllt und die Kapazitäten $0 \leq u_{ij} < \infty$ beschränkt sind. Dann hat das Maximale Fluss Problem eine optimale Lösung, denn in diesem Fall ist die Lösungsmenge nicht-leer und kompakt.

2.4.2 Minimaler Schnitt und Maximaler Fluss

Definition 2.4.5.

- $C \subset V$ ist ein Schnitt im Netzwerk N , wenn $Q \in C$ und $S \notin C$.
- Die Kapazität von einem Schnitt C ist der Wert.

$$\sum_{j \in C, k \notin C} u_{jk}$$

Beispiel 2.4.6.

Die Kapazität vom Schnitt in Abbildung 2.9 ist : $2 + 4 + 5 = 11$.

Abbildung 2.9: *Kosten von einem Schnitt*

Korollar 2.4.7.

Sei C ein Schnitt und x ein Fluss mit Wert w . Dann gilt

$$w = \sum_{j \in C, k \notin C} x_{jk} - \sum_{i \notin C, j \in C} x_{ij}$$

Beweis.

Übung.

Mit Korollar 2.4.7 erhalten wir sofort die folgende Abschätzung

$$w = \sum_{j \in C, k \notin C} x_{jk} - \sum_{i \notin C, j \in C} x_{ij} \leq \sum_{j \in C, k \notin C} u_{jk}.$$

$\underbrace{\hspace{1.5cm}}_{x_{jk} \leq u_{jk} \text{ und } x_{ij} \geq 0}$

Also der Wert vom Fluss x ist beschränkt durch die Kapazität vom Schnitt.

Bemerkung 2.4.8.

Wenn $C = Q$ und $C = V \setminus \{S\}$ im Korollar 2.4.7, dann erhält man genau die Definition vom Wert eines Flusses. $(\sum x_{Qj})$

Theorem 2.4.9 (Maximale Fluss und minimaler Schnitt).

Jeder maximale Fluss hat genau eine der beiden Eigenschaften:

- *Es existiert ein zulässiger Fluss mit beliebig großen Wert und jeder Schnitt hat unbeschränkte Kapazität.*
- *Es existiert ein maximaler Fluss dessen Wert gleich der minimalen Kapazität eines Schnittes ist.*

Beweis.

Korte und Vygen.

Theorem 2.4.10 (Maximale Fluss und minimaler Schnitt).

Wenn $u \mapsto \mathbb{N}_0$, dann existiert ein ganzzahliger maximaler Fluss.

2.4.3 Algorithmus von Ford und Fulkerson

Idee

- Der Ford und Fulkerson Algorithmus berechnet einen maximalen Fluss in N zwischen Q und S .
- Die Methode beruht auf der Annahme, daß ein Fluss maximal ist, wenn es keinen flußerhöhenden Pfad zwischen Q und S gibt.

Definition 2.4.11 (brauchbarer Pfad, flußerhöhender Pfad).

Sei x ein Fluss mit Wert w

- Sei ein $v_0 - v_r$ Pfad $(v_0, \dots, v_r)$ im unterliegenden ungerichteten Netzwerk gegeben. Der pfad heißt **brauchbar**, wenn $x_{ij} < u_{ij}$ gilt für alle Vorwärtskanten und $x_{ij} > 0$ für alle Rückwärtskanten (R-Kante).

- Eine Kante (i, j) heißt Vorwärtskante (V-Kante), wenn $i = v_k$ und $j = v_{k+1}$ gilt für ein k .
- Eine Kante (i, j) heißt Rückwärtskante (R-Kante), wenn $i = v_{k+1}$ und $j = v_k$ gilt für ein k .
- Ein brauchbarer $Q - S$ Pfad heißt flußerhöhender Pfad.

Beispiel 2.4.12 (brauchbarer Pfad).

Siehe Abbildung 2.10.

Abbildung 2.10: *Beispiele V-Kante und R-Kante*

Es gelten folgende Eigenschaften für den Ford-Fulkerson Algorithmus

- W ist ein flußerhöhender Pfad, dann gilt $d > 0$
- $\bar{x}$ ist ein zulässiger Fluß denn die Erhaltungsgleichungen sind erfüllt und d ist so gewählt, daß $0 \leq x_{ij} \leq x_{ij} + d \leq u_{ij}$ für V-Kanten und $u_{ij} \geq x_{ij} \geq x_{ij} - d \geq 0$ für R-Kanten
- Sei $\bar{w}$ der Wert von x .

Es fehlt uns nun noch eine Prozedur zur Konstruktion eines flußerhöhenden Pfades. Ford-Fulkerson haben einen Markierungsalgorithmus vorgeschlagen, welcher die Knoten in eine markierte Knotenmenge und eine unmarkierte

Algorithmus : Ford-Fulkerson Algorithmus

Daten : $G = (V, E, c)$

Ergebnis : Einen maximalen Fluss $\hat{x}$

Initialisierung : Sei x ein zulässiger Fluss

zBsp. $x_{ij} = 0$

solange *Finde einen flußerhöhenden Pfad W* **tue**
 wenn *kein flußerhöhenden Pfad W gefunden* **dann**
 STOP ;
 x ist maximaler Fluß
 Ende
 Berechnen den Fluß $\bar{x}$ wie folgt

$$\bar{x}_{ij} = \begin{cases} x_{ij} + d & (i, j) \text{ ist V-Kante in } W \\ x_{ij} - d & (i, j) \text{ ist R-Kante in } W \\ x_{ij} & (i, j) \text{ ist nicht in } W \end{cases}$$

$d = \min\{d_v, d_r\}$;
 $d_v = \min\{u_{ij} - x_{ij} \mid (i, j) \text{ ist V-Kante in } W\}$;
 $d_r = \min\{x_{ij} \mid (i, j) \text{ ist R-Kante in } W\}$;
 wenn $d = \infty$ **dann**
 STOP ;
 x ist unbeschränkt.
 Ende
Ende

Algorithmus 5: Ford-Fulkerson Algorithmus

Knotenmenge unterteilt. C ist dann die markierte Knotenmenge. Die Menge C wird weiter in 'besucht' und 'unbesucht' unterteilt. Das 'Besuchen' eines markierten Knoten $i \in C$ bedeutet

- $\forall(i, j)$, wenn $x_{ij} < u_{ij}$ und $j \notin C$ dann markiere j und setze $C = C \cup \{j\}$.
- $\forall(j, i)$, wenn $x_{ij} > 0$ und $j \notin C$ dann markiere j und setze $C = C \cup \{j\}$.

Es gelten folgende Eigenschaften für den Ford-Fulkerson Markierungsalgorithmus

- Der Algorithmus endet ein flußerhöhenden Pfad gefunden ist oder es keinen brauchbaren $Q - S$ Pfad gibt.

Algorithmus : Ford-Fulkerson Markierungsalgorithmus

Daten : $G = (V, E, c)$

Ergebnis : Einen maximalen Fluss $\hat{x}$

Initialisierung : Markiere Q und setze $C = \{Q\}$

solange *Nicht alle markierten Knoten besucht wurden* **tue**

Wähle $i \in C$ mit i nicht besucht. ;

Besuche i ;

wenn S ist markiert **dann**

STOP ;

Ende

Ende

Algorithmus 6: Ford-Fulkerson Markierungsalgorithmus

- Wenn es keinen brauchbaren $Q - S$ Pfad gibt, dann definieren die markierten Knoten in C den Schnitt, weil $Q \in C$ und $S \notin C$.

Beispiel 2.4.13.

Möglichst viele Passagiere müssen von Q nach S transportiert werden. Allerdings gibt es keine direkte Flugverbindungen zwischen den Städten und die Passagiere müssen daher über Umwege fliegen. Allerdings haben die Anschlußflüge nur eine beschränkte Anzahl an Restplätzen. Ziel ist es eine maximale Anzahl an Passagieren zu befördern, damit die Fluglinie die Entschädigungen minimiert. Es liegt auf der Hand, daß dieses Problem sich anhand der Algorithmus von Ford und Fulkerson lösen lässt.

Im folgenden zeigen wir unter welchen Annahmen der Algorithmus konvergiert. Keine Konvergenz erhalten wir, wenn wir keine Annahmen bezüglich der Ordnung der Knoten treffen, da dies zu Zyklen führen kann.

Theorem 2.4.14.

Es gelten folgende Annahmen

- i) Jedes u_{ij} ist endlich und positiv ganzzahlig.*
- ii) Mindestens ein Schnitt C hat endliche Kapazität M .*
- iii) Die Initialisierung vom Fluß ist ganzzahlig.*

Der Ford-Fulkerson Algorithmus terminiert nach höchstens $M + 1$ Iterationen.

Abbildung 2.11: *Kosten von einem Schnitt*

Beweis.

Der Initialfluß ist ganzzahlig. Dann wird in jedem Schritt ein ganzzahliger Fluß generiert weil $d > 0$ ganzzahlig ist. Das impliziert, daß der Wert vom Fluß um mindestens 1 erhöht wird. Dann wissen wir, daß der Fluss am Beginn der k -ten Iteration mindestens den Wert $k - 1$ hat. Laut Annahme wissen wir, daß jeder Wert eines möglichen Flusses von oben beschränkt ist durch die Kapazität M vom Schnitt. Dann wissen wir die k -te Iteration wird nur durchgeführt wenn $k - 1 < M$. Letztlich wissen wir das der Algorithmus höchstens $M + 1$ Iteration durchführt.

Bemerkung 2.4.15.

- Wenn die Annahmen nicht erfüllt sind, dann kann Algorithmus 6 versagen.
- Es existieren Erweiterungen des Ford-Fulkerson Algorithmus, welche mit unendlich und nicht ganzzahligen Kapazitätsschranken konvergieren. Mehr zu dem Thema finden sie im Buch von Korte und Vygen (Theorem 8.14)

2.5 Dynamische Programmierung

Viele kombinatorische Optimierungsproblem lassen sich umschreiben auf dynamische Optimierungsprobleme. Unter anderem finden sich folgende und einige mehr

- Lagerproblem
- Das Knapsackproblem
- Zuordnungsproblem
-

Abbildung 2.12: *Diskret dynamisches Optimierungsproblem.*

2.5.1 Dynamisches Optimierungsproblem

Folgende Zutaten benötigen wir für die Formulierung eines diskreten Optimierungsproblem:

- $y(t_j)$ sind die Zustandsvariablen eines zeitabhangigen Systems zum Zeitpunkt t_j .
- $u(t_j)$ sind die Kontrollvariablen zum Zeitpunkt t_j . Sie beeinflussen den Zustand zum Zeitpunkt t_j .

Abbildung 2.13: *Beispiele diskreter Pfade für unterschiedliche Kontrollfunktionen.*

- Die diskrete dynamische Gleichung $y(t_{j+1}) = g(t_j, y(t_j), u(t_j))$ beschreibt den Übergang vom Zustand $y(t_j)$ in den Zustand $y(t_{j+1})$. Die Übergangsfunktion g hängt von Zeit, Zustand und Kontrolle ab.
- Ein zu minimierendes Zielfunktional.
- Restriktionen auf Zustandsvariablen und Kontrollvariablen.

Im folgenden zwei schematische Darstellungen eines diskreten dynamischen Systems.

2.5.2 Zwei Beispiele

Die folgenden zwei Beispiele veranschaulichen, wie die Modellierung auf diskrete dynamische Optimierungsprobleme führen.

Lagerproblem

Ein Unternehmen besitzt ein Lager und möchte eine kostenoptimalen Lagerplanung aufstellen für eine endliche Zeitperiode $t_0 < t_1 < \dots < t_N$. Ein Produkt kann in dem Lager in diesem Zeithorizont zwischengelagert werden bevor es zum Verkauf ausgeliefert wird. Sei $u_j \geq 0$ ein Zulieferung zum Zeitpunkt t_j , $r_j \geq 0$ die Nachfrage in der Periode $[t_j, t_{j+1})$ und y_j die Anzahl der

aktuell lagernden Produkte im Lager (kurz bevor der Auslieferung). Dann ergibt sich folgende dynamische Gleichung

$$y_{t+1} = y_j + u_j - r_j, \quad j = 0, \dots, N-1.$$

Zusätzlich verlangen wir, dass die Nachfrage immer gedeckt ist, also $y_{j+1} \geq 0$ für alle Zeiten. O.B.d.A. setzen wir die Randbedingungen $y_0 = y_N = 0$ voraus, also der Lagerbestand zu Anfang und Ende ist 0. Die Zustellungskosten zum Zeitpunkt t_j sind durch folgende Gleichung beschrieben

$$B(u_j) = \begin{cases} K + cU_j, & \text{wenn } u_j > 0, \\ 0 & \text{sonst.} \end{cases}.$$

K sind die Fixkosten für einen Transport und c sind die Kosten pro Einheit. Die Lagerkosten werden zum Ende einer Zeitperiode schlagend und sind durch hy_{j+1} beschrieben, wobei h die Kosten pro Einheit sind. Somit erhalten wir Gesamtkosten in der Höhe von

$$\sum_{j=0}^{N-1} (K\delta(u_j) + cu_j + hy_{j+1}),$$

mit

$$\delta(u_j) = \begin{cases} 1, & \text{wenn } u_j > 0, \\ 0 & \text{sonst,} \end{cases}.$$

Zusätzlich gelten folgende Gleichungen wie man sich leicht überlegen kann:

$$\begin{aligned} \sum_{j=0}^{N-1} u_j &= \sum_{j=0}^{N-1} (y_{j+1} - y_j + r_j) = (y_N - y_0) + \sum_{j=0}^{N-1} r_j = \sum_{j=0}^{N-1} r_j \\ \sum_{j=0}^{N-1} y_{j+1} &= \sum_{j=0}^{N-1} (y_j + u_j - r_j) = \sum_{j=0}^{N-1} y_j + \sum_{j=0}^{N-1} u_j - \sum_{j=0}^{N-1} r_j = \sum_{j=0}^{N-1} y_j \end{aligned}$$

Somit lässt sich das Lagerproblem als diskretes dynamisches Optimierungsproblem aufschreiben wie im Eingang des Kapitels besprochen:

$$\text{Minimize } \sum_{j=1}^N (K\delta(u_j)hy_{j+1})$$

$$y_{t+1} = y_j + u_j - r_j, \quad j = 0, \dots, N-1,$$

$$y_0 = y_N = 0,$$

$$y_j \geq 0, \quad j = 0, \dots, N,$$

$$u_j \geq 0, \quad j = 0, \dots, N,$$

Abbildung 2.14: *Lagerproblem.*

Knapsack

Gegeben sind ein Knapsack mit Volume A und N Objekte. Objekt i hat ein Gewicht a_i und einen Wert c_i . Nun soll der Knapsack so gepackt werden, daß der Wert maximal wird und das Gewicht nicht das zulässige Volumen überschreitet. Das führt auf folgendes Optimierungsproblem

$$\begin{aligned} &\text{Maximiere } \sum_j^N c_j u_j \\ &\text{NB } \sum_j^N a_j u_j \leq A \\ &u_j \in \{0, 1\}, \quad j = 1, \dots, N, \\ &u_j = \begin{cases} 1, & \text{wenn } j \text{ im Knapsack,} \\ 0 & \text{sonst.} \end{cases} \end{aligned}$$

Das vorliegende Optimierungsproblem ist linear und ganzzahlig. Wir transformieren das Problem in ein äquivalentes diskret dynamisches Optimierungsproblem. Sei y_j das restliche Volume nach j Schritten, wobei die Objekte $i = 1, \dots, j - 1$ eingepackt wurden oder nicht. Dann erhalten wir folgendes

dynamisches Optimierungsproblem

$$\begin{aligned}
 &\text{Maximiere } \sum_{j=1}^N c_j u_j \\
 &\text{NB } y_{j+1} = y_j - a_j u_j, \quad j = 1, \dots, N, \\
 &\quad y_1 = A, \\
 &\quad 0 \geq y_j, \quad j = 1, \dots, N, \\
 &\quad u_j = \begin{cases} \in \{0, 1\}, & \text{wenn } y_j \geq a_j, \\ 0 & \text{wenn } y_j \leq a_j \end{cases} \quad j = 1, \dots, N.
 \end{aligned}$$

Dann ist y_{N+1} der verbleibende Platz im Knapsack.

2.5.3 Formale Formulierung eines diskret dynamischen Optimierungsproblem

Wir definieren folgende Objekte

- $\mathbb{G} := \{t_j | j = 0, \dots, N\}$... Gitter mit $N + 1$ fixen (Zeit)Punkten $t_0 < \dots, t_N$.
- Zustandsfunktion $y : \mathbb{G} \mapsto \mathbb{R}^n, t_j \mapsto y(t_j)$,
- Kontrollfunktion $u : \mathbb{G} \mapsto \mathbb{R}^m, t_j \mapsto u(t_j)$,
- Zielfunktion $\sum_{j=0}^N f(t_j, y(t_j), u(t_j))$ mit $f : \mathbb{G} \times \mathbb{R}^n \times \mathbb{R}^m \mapsto \mathbb{R}$.
- Dynamische Gleichung $y(t_{j+1}) = g(t_j, y(t_j), u(t_j)), j = 0, \dots, N - 1$ und $g : \mathbb{G} \times \mathbb{R}^n \times \mathbb{R}^m \mapsto \mathbb{R}^n$.
- Zustandsnebenbedingungen $y(t_j) \in Y(t_j) \subset \mathbb{R}^n, j = 0, \dots, N$. (nicht-leer)
- Kontrollnebenbedingungen $u(t_j) \in U(t_j, y(t_j)) \subset \mathbb{R}^m, j = 0, \dots, N$. (nicht-leer)

Das Ziel ist die Zielfunktion bzg y und u zu minimieren unter der Nebenbedingung, daß die dynamische Gleichung und die Nebenbedingungen erfüllt sind. So erhalten wir folgendes mathematisches Optimierungsproblem:

Diskretes dynamisches Optimierungsproblem (DOP):

$$\begin{array}{ll}
 \text{Minimiere} & \sum_{j=0}^N f(t_j, y(t_j), u(t_j)) \\
 \text{bzg} & y \in \{y | y : \mathbb{G} \mapsto \mathbb{R}^n\}, u \in \{u | u : \mathbb{G} \mapsto \mathbb{R}^m\} \\
 \text{NB} & \begin{aligned}
 y(t_{j+1}) &= g(t_j, y(t_j), u(t_j)), \quad j = 1, \dots, N-1, \\
 y(t_j) &\in Y(t_j), \quad j = 1, \dots, N, \\
 u(t_j) &\in U(t_j, y(t_j)), \quad j = 1, \dots, N.
 \end{aligned}
 \end{array}$$

Bemerkung 2.5.1. FaSt immer sind die zulässigen Mengen $Y(t_j)$ und $U(t_j, y)$ durch Ungleichungen und Gleichungen definiert.

- $Y(t_j) = \{y \in \mathbb{R}^n | r(t_j, y) \leq 0, h(t_j, y) = 0\}$.
- $U(t_j, y) = \{u \in \mathbb{R}^m | \tilde{r}(t_j, y, u) \leq 0, \tilde{h}(t_j, y, u) = 0\}$.
- Die Kotrollfunktion wird oft auch nur über "Box constraints" definiert:

$$u(t_j, y) \in \{v = (v_1, \dots, v_m)^T \in \mathbb{R}^m | a_j \leq v_j \leq B_j, j = 1, \dots, m\}.$$

2.5.4 Dynamische Programmierung und das Bellman Optimalitätsprinzip

Die dynamische Programmierung basiert auf dem Bellman schen Optimalitätsprinzip. Dynamische Programmierung ist eine der ersten Methoden zum Lösen von DOP. Es gibt eine Fülle an Literatur die sich dem Thema Dynamische Programmierung widmen(c.f. TODO).

Bellman's Optimalitätsprinzip

Sei $t_k \in \{t_0, \dots, t_N\}$ ein fixer Zeitpunkt, $\mathbb{G}_k := \{t_j | j = k, \dots, N\}$ und $\hat{y} \in Y(t_k)$ ein zulässiger Zustand. Betrachte das

Diskrete dynamische Optimierungsproblem ($P(t_k, \hat{y})$):

$$\begin{array}{ll}
 \text{Minimiere} & \sum_{j=0}^N f(t_j, y(t_j), u(t_j)) \\
 \text{bzg} & y \in \{y | y : \mathbb{G} \mapsto \mathbb{R}^n\}, u \in \{u | u : \mathbb{G} \mapsto \mathbb{R}^m\} \\
 \text{NB} & y(t_{j+1}) = g(t_j, y(t_j), u(t_j)), \quad j = 1, \dots, N-1, \\
 & y(t_j) \in Y(t_j), \quad j = 1, \dots, N, \\
 & u(t_j) \in U(t_j, y(t_j)), \quad j = 1, \dots, N.
 \end{array}$$

Definition 2.5.2 (Optimalitätsfunktion). Sei $t_k \in \mathbb{G}$. Für $\hat{y} \in Y(t_k)$, sei $V(t_k, \hat{y})$ der optimale Wert vom Problem $P(t_k, \hat{y})$. Für $\hat{y} \notin Y(t_k)$ setzen wir $V(t_k, \hat{y}) = \infty$. Die Optimalitätsfunktion ist $V : \times \mathbb{R}^n \mapsto \bar{\mathbb{R}}, (t_k, \hat{y}) \mapsto V(t_k, \hat{y})$.

Theorem 2.5.3. Sei $\hat{y}(\cdot)$ und $\hat{u}(\cdot)$ eine optimale Lösung von (DOP). Dann gilt $\hat{y}|_{\mathbb{G}_k}$ und $\hat{u}|_{\mathbb{G}_k}$ sind optimale Lösung von $(P(t_k, \hat{y}))$.

Beweis.

- Angenommen $\hat{y}|_{\mathbb{G}_k}$ und $\hat{u}|_{\mathbb{G}_k}$ sind nicht optimal.
- Dann existieren $\tilde{y}|_{\mathbb{G}_k}$ und $\tilde{u}|_{\mathbb{G}_k}$ mit der Eigenschaft

$$\sum_{j=k}^N f(t_j, \tilde{y}(t_j), \tilde{u}(t_j)) < \sum_{j=k}^N f(t_j, \hat{y}(t_j), \hat{u}(t_j)).$$

- Dann folgt

$$\sum_{j=0}^{k-1} f(t_j, \hat{y}(t_j), \hat{u}(t_j)) + \sum_{j=k}^N f(t_j, \tilde{y}(t_j), \tilde{u}(t_j)) < \sum_{j=0}^N f(t_j, \hat{y}(t_j), \hat{u}(t_j)).$$

- Das ist ein Widerspruch zur Optimalität von $\hat{y}(\cdot)$ und $\hat{u}(\cdot)$.

Bemerkung 2.5.4. Das Optimalitätsprinzip besagt, daß die Entscheidungen in den Perioden $k, k+1, \dots, N$ von (DOP) für einen Zustand y_k unabhängig von den Entscheidungen in den Perioden $t_0, \dots, t_{k-1}$ sind.

- Das Optimalitätsprinzip erlaubt das (DOP) in unabhängige Teilprobleme zu zerlegen.
- Analog lässt sich sagen die Zielfunktion ist separierbar und die Nebenbedingungen wirken auf y und u nur zum Zeitpunkt t_j .

Dynamische Programmierung

Anhand des Optimalitätsprinzips lässt sich eine Rekursion für die Optimalitätsfunktion herleiten. Es gelte folgende Konvention : $f(t_j, y, u) = \infty$, wenn $y \notin Y(t_j)$.

Wir wissen, daß die Optimalitätsfunktion gegeben ist durch

$$V(t_N, y) = \min_{u \in U(t_N, y)} f(t_N, y, u). \quad (2.1)$$

Nehmen wir nun an, daß wir die Optimalitätsfunktion $V(t_{j+1}, y)$ für jedes $y \in \mathbb{R}^n$ kennen. Dann folgt mit dem Optimalitätsprinzip

$$V(t_N, y) = \min_{u \in U(t_N, y)} \{f(t_N, y, u) + V(t_{j+1}, g(t_j, y, u))\}, \quad j = 0, \dots, N-1. \quad (2.2)$$

Mit 2.1 und 2.2 berechnen wir die Optimalitätsfunktion rekursiv von t_N zum Anfangszeitpunkt. So erhalten wir den optimalen Initialzustand $\hat{y}(t_0)$ von (DOP) anhand von

$$\hat{y}(t_0) = \arg \min_{y \in Y(t_0)} V(t_0, y). \quad (2.3)$$

Gleichungen 2.1, 2.2 und 2.3 bilden die Grundlagen für folgende dynamische Programmiermethoden.

Bemerkung 2.5.5. Beide Programmiermethoden berechnen optimale Lösung für das (DOP). Erstere ist vorzuziehen für eine Computerimplementierung, da die optimale Kontrolle, als Funktion von Zustand und Zeit, nicht extra gespeichert werden muss. Zweitere ist vorzuziehen wenn mit der Papier und Bleistift gerechnet wird.

Algorithmus : Dynamische Programmiermethode 1

i) Rückwärtsberechnung

Sei $V(t_N, y)$ gegeben durch 2.1;

für alle $j = N - 1, \dots, 0$ **tue**

$V(t_j, y)$ wie in 2.2;

Ende

ii) Vorwärtswärtsberechnung;

Sei $\hat{y}(t_0)$ gegeben durch 2.3;

für alle $j = 0, \dots, N - 1$ **tue**

$\hat{u}(t_j) = \arg \min_{u \in U(t_j, \hat{y}(t_j))} \{f(t_j, \hat{y}(t_j), u) + V(t_{j+1}, g(t_j, \hat{y}(t_j), u))\};$

$\hat{y}(t_{j+1}) = g(t_j, \hat{y}(t_j), \hat{u}(t_j));$

Ende

$\hat{u}(t_N) = \arg \min_{u \in U(t_N, \hat{y}(t_N))} \{f(t_N, \hat{y}(t_N), u)\}$

Algorithmus 7: Dynamische Programmiermethode 1

Abbildung 2.15: *Bellman's Optimalitätsprinzip. Teiltrajektorien sind optimal.*

Implementierung

Im folgenden erarbeiten wir einen implementierbaren Algorithmus für das folgende diskrete dynamische Optimierungsproblem:

$$\begin{aligned} \text{Minimiere} \quad & \sum_{j=0}^N f(t_j, y(t_j), u(t_j)) \\ \text{NB} \quad & y(t_{j+1}) = g(t_j, y(t_j), u(t_j)), \quad j = 1, \dots, N-1, \\ & y(0) = y_a, \\ & y(t_j) \in [y_l, y_u], \quad j = 1, \dots, N, \\ & u(t_j) \in [u_l(t_j, y(t_j)), u_u(t_j, y(t_j))], \quad j = 0, \dots, N. \end{aligned}$$

Abbildung 2.16: *Dynamische Programmierung und Rekursion der Optimalitätsfunktion.*

Algorithmus : Dynamische Programmiermethode 2

i) Rückwärtsberechnung

Sei $V(t_N, y)$ gegeben durch 2.1 und $u^*(t_N, y)$ die entsprechende optimale Kontrolle;

für alle $j = N - 1, \dots, 0$ **tue**

$V(t_j, y)$ wie in 2.2;

 Sei $u^*(t_j, y)$ die entsprechende optimale Kontrolle zum Zeitpunkt t_j für y .

Ende

ii) Vorwärtswärtsberechnung;

Sei $\hat{y}(t_0)$ gegeben durch 2.3;

für alle $j = 0, \dots, N - 1$ **tue**

$\hat{u}(t_j) = u^*(t_j, \hat{y}(t_j))$ $\hat{y}(t_{j+1}) = g(t_j, \hat{y}(t_j), \hat{u}(t_j))$;

Ende

$\hat{u}(t_N) = u^*(t_N, \hat{y}(t_N))$

Algorithmus 8: Dynamische Programmiermethode 2

Das Intervall $[y_l, y_u]$ ist in ein äquidistantes Gitter mit M Teilintervallen der Länge $h = (y_u - y_l)/M$

$$Y = \{y_l + i\dot{h} | i = 0 \dots M\}.$$

Y ist der Zulässigkeitsbereich der Zustandsvariable y . Analog diskretisieren wir die Kontrollregion $[u_l(t_j, y(t_j)), u_u(t_j, y(t_j))]$ in M gleichgroße Bereiche der Länge $h_j = (u_u(t_j, y(t_j)) - u_l(t_j, y(t_j)))/M_j$

$$U(t_j, y(t_j)) = \{u_l(t_j, y(t_j)) + ih_j | i = 0 \dots M_j\}.$$

$U(t_j, y(t_j))$ ist der Zulässigkeitsbereich der Kontrollvariable u im Schritt j .

Algorithmus : Bellman'sche dynamische Programmiermethode

i) Rückwärtsberechnung

für alle $y \in Y$ **tue**

$$V(t_N, y) = \min_{u \in U(t_N, y)} f(t_N, y, u).$$

Ende

für alle $j = N - 1, \dots, 0$ **tue**

für alle $y \in Y$ **tue**

$$V(t_j, y) = \min_{u \in U(t_j, y); g(t_j, y, u) \in [y_l, y_u]} \{f(t_j, y, u) + V(t_{j+1}, g(t_j, y, u))\};$$

Ende

Ende

ii) Vorwärtswertsberechnung;

Sei $\hat{y}(1) = y_a$;

für alle $j = 0, \dots, N - 1$ **tue**

$$\begin{aligned} \hat{u}(t_j) = & \arg \min_{u \in U(t_j, \hat{y}(t_j)); g(t_j, \hat{y}(t_j), u) \in [y_l, y_u]} \{f(t_j, \hat{y}(t_j), u) + V(t_{j+1}, g(t_j, \hat{y}(t_j), u))\}; \\ \hat{y}(t_{j+1}) = & g(t_j, \hat{y}(t_j), \hat{u}(t_j)); \end{aligned}$$

Ende

$$\hat{u}(t_N) = \arg \min_{u \in U(t_N, \hat{y}(t_N))} \{f(t_N, \hat{y}(t_N), u)\}$$

Algorithmus 9: Bellman'sche dynamische Programmiermethode

Bemerkung 2.5.6. Die Berechnung von $V(t_j, y)$ und $\hat{u}(t_j)$ im Algorithmus 9 verwendet die Werte $V(t_{j+1}, y(t_{j+1}))$ mit $y(t_{j+1}) = g(t_j, y, u) \in [y_l, y_u]$. Wenn $y(t_{j+1})$ nicht genau einen Gitterpunkt in Y trifft, dann berechnen wir die den Wert der Optimalitätsfunktion zum Zustand $y(t_{j+1})$ anhand von linearer Interpolation.

Zum Abschluß bemerken wir, daß die dynamische Programmierung schlecht skaliert mit der Dimension des Problems. Dieses Verhalten wir oft auch als

Fluch der Dimensionalität bezeichnet. Dies sieht man sofort, bei der Berechnung Optimalitätsfunktion die für alle Zeiten und Zustände ausgewertet wird. Mit großen N wird dies sehr langsam. Aber in den meisten Anwendungen die wir im Eingang des Skriptums kennengelernt haben und speziell für Probleme mit ganzzahligen Daten ist die Dynamische Programmiermethodik oft sehr effizient.

Kapitel 3

Lineare Programmierung

Im vorliegenden Skriptum beschränken wir uns auf lineare Programme in Standardform. Das ist keinerlei eine Einschränkung der, weil jedes lineare Programm auf Standardform gebracht werden kann. Die Transformationstechniken können sie in jedem Buch zur Theorie linearer Programme nachlesen. Deswegen gehen wir im folgenden davon aus, daß der Leser die entsprechenden Vorkenntnisse mitbringt.

3.1 Lineare Programme und der Simplex Algorithmus

Definition 3.1.1 (Lineares Programm in Standardform).

Sei

$$c = \begin{pmatrix} c_1 \\ \vdots \\ c_n \end{pmatrix} \in \mathbb{R}^n, \quad b = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix} \in \mathbb{R}^m, \quad A = \begin{pmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{pmatrix} \in \mathbb{R}^{m \times n}$$

mit $\text{Rang}(A) = m$ gegeben. Dann ist das lineare Programm wie folgt definiert:

$$\min c^T x$$

unter den Nebenbedingungen

$$Ax = b \quad \text{und} \quad x \geq 0;$$

Definition 3.1.2 (Zielfunktion, zulässige Menge, zulässige Punkte, Optimalität).

i) $f(x) = c^T x$ ist die Zielfunktion.

ii) Die zulässige Menge ist

$$M := \{x \in \mathbb{R}^n | Ax = b, x \geq 0\}.$$

iii) $x \in M$ heißt zulässige Vektor.

iv) $\hat{x} \in M$ heißt optimal, wenn

$$c^T \hat{x} \leq c^T x \quad \forall x \in M.$$

Die zulässige Menge M ist der endliche Durchschnitt der affinen Menge $\{x \in \mathbb{R}^n | Ax = b\}$ und der Menge $\{x \in \mathbb{R}^n | x \geq 0\}$. Diese Menge wird auch Polyeder genannt. In der linearen Programmierung ist der Polyeder konvex und hat Ecken. Die Ecken des Polyeders spielen eine wichtige Rolle. Das folgende Theorem sagt aus, daßes ausreicht nur die Ecken des Polyeders M um **eine** optimale Lösung zu finden.

Theorem 3.1.3.

Sei LP aus 3.2.2 mit $M \neq \emptyset$, dann gilt

- Entweder $f(x)$ ist nach unten unbeschränkt über M oder LP (3.2.2) hat eine optimale Lösung und mindestens eine Ecke von M ist Element der optimalen Lösungen.
- M ist beschränkt, dann existiert eine optimale Lösung $x \in M$, genau dann wenn x eine konvex Kombination optimaler Ecken ist.

Die geometrische Charakterisierung einer Ecke eines Polyeder ist : $x \in M$ ist eine Ecke von M genau dann wenn x als konvex Kombination zulässiger Punkte aus M angeschrieben werden kann. Die folgende äquivalente Charakterisierung von Ecken einer zulässigen Lösung ist geeigneter für numerische Berechnungen. Wir führen die folgende Notation ein:

- Die Spalten von A werden mit $a^j : (a_{1,j}, \dots, a_{m,j})^T$ bezeichnet.
- Sei B eine Indexmenge, dann ist x_B ein Vektor mit Komponenten x_i und $i \in B$.
- Analog definieren wir die Matrix A_B mit bestehend aus den Spalten a^i und $i \in B$.

Definition 3.1.4 (zulässige Basislösung).

Sei $A \in \mathbb{R}^{m \times n}$, $\text{Rang}(A)=m$, $b \in \mathbb{R}^m$ und $M = \{x \in \mathbb{R}^n | Ax = b\}$. Sei $B \subseteq \{1, \dots, n\}$ eine Indexmenge mit $|B| = m$, $N = \{1, \dots, n\} \setminus B$ und die Spalten a^i mit $i \in B$ sind linear unabhängig. Dann ist x eine Basislösung von M , wenn

$$A_B x_B = b, \quad x_N = 0.$$

Ausserdem ist x eine zulässige Basislösung von M , wenn x Basislösung und $x_B \geq 0$.

Theorem 3.1.5.

$x \in M$ ist eine Ecke von M , genau dann wenn x eine zulässige Lösung von M ist.

Definition 3.1.6 (Zielfunktion, zulässige Menge, zulässige Punkte, Optimalität).

(Basis) Sei $\text{Rang}(A) = m$ und x eine zulässige Basislösung von LP. Jedes System von m linear unabhängigen Spalten a^i ist eine Basis von x .

(Nicht Basis) Sei a^i mit $i \in B$ eine Basis von x . B ist die Basis-Indexmenge und $N := \{1, \dots, n\} \setminus B$ ist die Nichtbasis-Indexmenge. Dann ist die Nichtbasis-Matrix gleich A_N . Ein Basisvektor ist x_B und ein Nichtbasis-Vektor ist x_N .

Mit der eben eingeführten Notation erhalten wir den Simplex Algorithmus, welcher erstmals 1947 von Dantzig eingeführt wurde.

Der Simplex Algorithmus 10 löst in jeder Iteration drei lineare Gleichungssysteme. Diese treten in den Teilschritten a), b) und d) auf.

Theorem 3.1.7 (Endliche Laufzeit).

Der Algorithmus 10 findet eine optimale Lösung oder detektiert, daß die Zielfunktion nach unbeschränkt ist. In beiden Fällen terminiert der Algorithmus nach endlich vielen Schritten

3.2 Dualität

Definition 3.2.1 (Primales Problem).

Sei $A \in \mathbb{R}^{m \times n}$ mit $\text{Rang}(A) = m$ gegeben. Dann ist das primale Problem gleich dem LP in Standardform:

$$\min c^T x \quad \text{NB.} \quad Ax = b \quad \text{und} \quad x \geq 0 \quad (\text{P})$$

Jedem primalen Problem kann ein anderes lineares Programm zugeordnet werden - das duale Problem.

Definition 3.2.2 (Duales Problem).

Das lineare Program

$$\max b^T \lambda \quad \text{NB.} \quad A^T \lambda \leq c \quad (\text{D})$$

heißt duales Problem von (P).

Das folgenden Dualisierungsverfahren lässt sich leicht anwenden unter der Konvention, daß das primale Problem das Minimierungsproblem ist.

Beispiel 3.2.3.

Geben sie das zu

$$\min -3x_1 - 4x_2 \quad 2x_1 + x_2 + x_3 = 8, \quad 4x_1 + x_2 + x_4 = 8, \quad x_i \geq 0.$$

duale Problem an.

Dasselbe Schema lässt sich komponentenweise anwenden mit anderen primalen Nebenbedingungen

$$x_i \begin{cases} \geq 0 \\ \leq 0 \\ \text{free} \end{cases}, i = 1, \dots, n, \quad \sum_{j=1}^n a_{ij} x_j \begin{cases} = \\ \leq \\ \geq \end{cases} b_i, i = 1, \dots, n.$$

Die duale Variable λ_i steht im Zusammenhang mit der i -ten primalen Nebenbedingung über das folgende Schema

$$\sum_{j=1}^n a_{ij} x_j \begin{cases} = \\ \leq \\ \geq \end{cases} b_i \quad \leftrightarrow \quad \lambda_i \begin{cases} \text{free} \leq 0 \\ \geq 0 \end{cases}$$

Und die i -te primal Variable x_i wird mit der i -ten Komponente der dualen Variabel über das folgende Schema identifiziert:

$$x_i \begin{cases} \geq 0 \\ \leq 0 \\ \text{free} \end{cases} \quad \leftrightarrow \quad (A^T \lambda)_i \begin{cases} \leq \\ \geq \\ = \end{cases} c_i.$$

$(A^T \lambda)_i$ steht für die i -te Komponente vom Vektor $(A^T \lambda)$, wobei A die Koeffizienten Matrix der primalen Nebenbedingungen ist. Und zuletzt wollen wir nochmal feststellen, daß dies unter der Konvention gültig ist, daß (P) ein Minimierungsproblem ist.

Theorem 3.2.4.

Die Dualisierung des dualen Problem führt auf das primale Problem zurück.

Mit diesem Theorem kann man frei zwischen dualen und primalen Problem hin und her springen. Nun folgen noch einige Zusammenhänge zwischen primalen und dualen Problem.

Theorem 3.2.5 (Schwache Dualität).

Sei x zulässig für (P) und sei λ zulässig für (D) . Dann gilt

$$b^T \lambda \leq c^T x.$$

Das Theorem der schwachen Dualität motiviert erst richtig das duale Problem. Durch duale zulässige Punkte erhält man untere Schranken für das Minimum des primalen Problems. Umgekehrt erhält man obere Schranken für das duale Problem anhand von zulässigen primalen Punkten. Diese Eigenschaft spielt für Verzweigungs-Methoden zur Lösung von ganzzahligen Programmen eine sehr wichtige Rolle. Ausserdem gilt folgendes Theorem

Theorem 3.2.6 (Hinreichendes Optimalitätskriterium).

Sei x zulässig für (P) und sei λ zulässig für (D) . Dann gilt

- i) Wenn $b^T \lambda = c^T x$, dann ist x optimal für (P) und λ ist optimal für (D) .*
- ii) $b^T \lambda = c^T x$ gilt genau dann wenn die folgende Bedingung vom komplementären Schlupf gilt :*

$$x_j \left(\sum_{i=1}^m a_{ij} \lambda_i - c_j \right) = 0, \quad j = 1, \dots, n.$$

Bemerkung 3.2.7. Die Bedingung vom komplementären Schlupf ist äquivalent mit dem folgendem: Sei $j = 1, \dots, n$, dann gilt

$$x_j > 0 \quad \Rightarrow \quad \left(\sum_{i=1}^m a_{ij} \lambda_i - c_j \right) = 0$$

und

$$\sum_{i=1}^m a_{ij} \lambda_i - c_j < 0 \quad \Rightarrow \quad x_j = 0$$

Das bedeutet: Entweder die primale Nebenbedingung ist aktiv $x_j \geq 0$ oder die duale Nebenbedingung $\sum_{i=1}^m a_{ij} \lambda_i \leq c_j$ ist aktiv. Es kann allerdings nicht sein, daß beide Nebenbedingungen, also primal und dual, inaktiv sind ($x_j > 0$ und $\sum_{i=1}^m a_{ij} \lambda_i < c_j$).

Theorem 3.2.8 (Starke Dualität).

(P) hat eine optimale Lösung x genau dann wenn (D) eine optimale Lösung λ besitzt. Ausserdem haben primale und duale Zielfunktion den gleichen optimalen Wert, wenn eine Lösung existiert

$$c^T x = b^T \lambda.$$

Die folgenden Eigenschaften erhalten wir mit 3.2.8 und 3.2.6

Korollar 3.2.9. *Sei x eine zulässige Lösung von (P) und sei λ eine zulässige Lösung von (D). Dann sind die folgenden Aussagen äquivalent*

- i) x ist optimal für (P) und λ ist optimal für (D).*
- ii) Es gilt $c^T x = b^T \lambda$.*
- iii) Bedingung vom komplementären Schlupf gilt.*

Bemerkung 3.2.10. Der Simplex Algorithmus 10 berechnet Ecken x die für (P) zulässig sind und duale Variablen λ , welche $c^T x = b^T \lambda$ erfüllen. Der Simplex Algorithmus terminiert sobald λ zulässig ist für (D). Mit anderen Worten ist die Zulässigkeit von λ für (D) das Optimalitätskriterium für Algorithmus 10.

3.3 Ein primal-dualer Algorithmus

Wir betrachten folgendes primales lineares Programm

$$(P) \quad \min c^T x \quad \text{NB} \quad Ax = b, \quad x \geq 0,$$

und das entsprechende duale Programm

$$(D) \quad \max b^T \lambda \quad \text{NB} \quad A^T \lambda \leq c, \quad \lambda \geq 0,$$

mit $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $x \in \mathbb{R}^n$ und $\lambda \in \mathbb{R}^m$. O.E.d.A. nehmen wir $b \geq 0$ an. Weiter haben wir die Spalten von A mit a^j .

Ziel ist die Konstruktion einer algorithmischen Methode zur Berechnung einer zulässigen Lösung x für (P) und einer zulässigen Lösung λ für (D), sodaß die Bedingung vom komplementären Schlupf gilt. Dann können wir aufgrund von Theorem 3.2.6 Optimalität für x und λ garantieren.

3.3.1 Konstruktion des Algorithmus

- Sei λ zulässig für (D) gegeben (wenn $c \geq 0$, wählen wir $\lambda = 0$, sonst müssen zuerst ein zusätzliches Problem lösen.)
- Definiere die Indexmenge

$$B := \{j \in \{1, \dots, n\} \mid (A^T \lambda)_j = c_j\}$$

der aktiven dualen Bedingungen.

- Bedingung vom komplementären Schlupf ist erfüllt, wenn wir ein zulässiges x für (P) finden mit

$$x_j \underbrace{((A^T \lambda)_j - c_j)}_{< 0 \text{ für } j \notin B} = 0, \quad \forall j \notin B$$

Wir müssen also ein $x \in \mathbb{R}^n$ mit folgenden Eigenschaften

$$\sum_{j \in B} a^j x_j = A_B x_b = b,$$

$$x_j \geq 0, \quad \forall j \in B, x_j = 0, \quad \forall j \notin B,$$

finden. Dieses Problem lässt sich durch folgendes lineares Hilfsprogramm lösen, welches wir schon beim Simplex Algorithmus 10 in ersten Schritt angewendet haben, um eine zulässige Basislösung zu berechnen. Das Problem erhalten wir durch die Einführung einer Schlupfvariablen $y \in \mathbb{R}^m$ mit nicht negativen Komponenten, was uns zum Restringierten primalen Problem führt

$$\min \sum_{i=1}^m y_i \tag{RP}$$

$$\begin{aligned} \text{NB} \quad & A_B x_B + y = b, \\ & x_j \geq 0, \quad j \in B, \\ & x_j = 0, \quad j \notin B, \\ & y_i \geq 0, \quad i = 1, \dots, m. \end{aligned}$$

(RP) kann zBsp mit dem Simplex Algorithmus gelöst werden und hat immer eine optimale Lösung, weil die Zielfunktion von unten durch $y = 0$ beschränkt ist auf der zulässigen Menge und $x = 0$ und $y = b \geq 0$ zulässig ist.

Sei f_{opt} der optimale Wert für (RP) und sei (x_{opt}, y_{opt}) die optimale Lösung von (RP).

- Unterscheidung von 2 Fällen
 - i) Wenn $f_{opt} = 0$, dann ist die zugehörige Lösung x_{opt} zulässig für (P) und erfüllt die Bedingung vom komplementären Schlupf. Und deswegen ist x_{opt} optimal für (P) und λ für (D).
 - ii) Wenn $f_{opt} > 0$, dann ist die zugehörige Lösung x_{opt} nicht zulässig für (P), weil $Ax_{opt} + y_{opt} = b$ mit y_{opt} ungleich dem Nullvektor. Wir müssen uns deswegen das duale Problem (RP) ansehen

$$\begin{array}{ll} \max b^T \lambda & \text{(DRP)} \\ \text{NB } A_B^T \lambda \leq 0, & \\ \lambda_j \geq 0, \quad i = j, \dots, m. & \end{array}$$

Sei $\bar{\lambda}$ eine optimale Lösung von (DRP). Aufgrund starker Dualität 3.2.8 haben wir $b^T \bar{\lambda} > 0$.

Um den Wert der dualen Zielfunktion zu verbessern wollen wir einen besseren dualen Vektor berechnen

$$\lambda^* = \lambda + t \hat{\lambda}$$

mit $t \in \mathbb{R}$. Der Wert der dualen Zielfunktion ist dann

$$b^T \lambda^* = b^T \lambda + t \underbrace{b^T \hat{\lambda}}_{>0}.$$

Wenn also $t > 0$ dann steigt auch der Wert der Zielfunktion. Zusätzlich wollen wir Zulässigkeit von λ^* bewahren, also

$$(A^T \lambda^*)_j = \underbrace{(A^T \lambda)_j}_{\leq c_j} + t(A^T \bar{\lambda})_j \leq c_j, \quad \forall j = 1, \dots, n.$$

Weil $\bar{\lambda}$, $(A_B^T \bar{\lambda}) \leq 0$, also $(a^j)^T \bar{\lambda} \leq 0$, erfüllt, ist die obere Ungleichung stetes erfüllt wenn $t \geq 0$ für $j \in B$.

Zuletzt müssen wir noch duale Zulässigkeit für $j \notin B$. Auch hier können wir zwei Fälle unterscheiden

- * a) Wenn $(A^T \bar{\lambda})_j \leq 0$ für $\forall j \notin B$, dann bleibt duale Zulässigkeit erhalten für $t \geq 0$. Für $t \rightarrow \infty$ ist die duale Zielfunktion nach oben unbeschränkt und hat keine Lösung. Und wegen starker Dualität 3.2.8 ist die primale Zielfunktion ebenfalls unbeschränkt.

- * b) Wenn $(A^T \bar{\lambda})_j > 0$ für $\forall j \notin B$, dann bleibt duale Zulässigkeit erhalten für t

$$t_{max} := \min \left\{ \frac{c_j - (A^T \bar{\lambda})_j}{(A^T \bar{\lambda})_j} \mid j \notin B, (A^T \bar{\lambda})_j > 0 \right\}.$$

Deswegen ist $\lambda^* = \lambda + t_{max} \bar{\lambda}$ zulässig für (D) und verbessert den Wert der Zielfunktion.

So kommen wir also zu folgenden Algorithmus

Beispiel 3.3.1. Wenden sie den primal-dualen Algorithmus auf folgendes LP an:

$$\min c^T x \quad \text{NB} \quad Ax = b, \quad x \geq 0,$$

mit

$$A = \begin{pmatrix} 1 & 2 & 1 & 0 \\ 2 & 1 & 0 & 1 \end{pmatrix} \quad c = \begin{pmatrix} -2 \\ -3 \\ 0 \\ 0 \end{pmatrix}, \quad b = \begin{pmatrix} 8 \\ 9 \end{pmatrix}.$$

Algorithmus : Simplex Algorithmus

Initialisierung : Transformiere das LP in Standardform

1)

Berechne eine zulässige Basislösung x für das LP mit Basisindexmenge B und Nicht-Basisindexmenge N . Dann sind A_B , x_B Basismatrix und Basisvektor und $x_N = 0$ der Nichtbasis-Vektor. ;

Existiert keine zulässige Lösung, dann: **STOP**

LP besitzt keine Lösung;

2)

a) **Berechne:**

$$\beta \text{ mit } A_B \beta = b$$

b) **Löse:**

$A_B^T \lambda = c_B$ und berechne $(\xi_j)_{j \in N}$

$$\xi^T = \lambda^T A_N - c_N^T.$$

c) **Überprüfe Optimalität:**

Wenn $\xi_j \leq 0$ für all $j \in N$, dann **STOP**. Die aktuelle Basislösung ist zulässig und optimal. Setze $x_B = \beta$, $x_N = 0$ und der Wert der Zielfunktion ist $C_B^T \beta$.

d) **Berechne Pivotspalte:**

Wähle einen Index q mit $\xi_q > 0$ und löse

$$A_B \gamma = a^q$$

mit a^q q te Spalte von A .

e) **Unbeschränktheit:**

Ist $\gamma \leq 0$, dann **STOP**

LP besitzt keine Lösung;

f) **Berechne Pivotzeile:**

Wähle einen Index i mit

$$\frac{\beta_i}{\gamma_i} = \min \left\{ \frac{\beta_i}{\gamma_i} \mid \gamma_i > 0, i \in B \right\}.$$

g) **Basiswechsel:**

Setze $B := (B \setminus \{i\}) \cup \{q\}$ und $N := (N \setminus \{q\}) \cup \{i\}$

h) Gehe zu a)

Algorithmus 10: Simplex Algorithmus

Algorithmus : Primal-dualer Algorithmus**1)**Sei λ zulässig für (D).**2)**Setze $B = \{j \in \{1, \dots, n\} | (A^T \lambda)_j = c_j\}$.**3)**Löse (RP) (zBsp mit Simplex). Sei f_{opt} der optimale Wert für die Zielfunktion und $\bar{\lambda}$ eine optimale Lösung von (DRP).**4)**Wenn $f_{opt} = 0$, dann **STOP**. x ist optimal für (P) und λ ist optimal für (D).**5)**Wenn $f_{opt} > 0$ und $(A^T \bar{\lambda})_j \leq 0$ für $\forall j \notin B$, dann **STOP**. (D) ist unbeschränkt und daher (P) nicht lösbar.**6)**

Berechne

$$t_{max} := \min \left\{ \frac{c_j - (A^T \bar{\lambda})_j}{(A^T \bar{\lambda})_j} \mid j \notin B, (A^T \bar{\lambda})_j > 0 \right\}.$$

Setze $\lambda = \lambda + t_{max} \bar{\lambda}$ und gehe zu **1**).**Algorithmus 11:** Primal-dualer Algorithmus

Kapitel 4

Ganzzahlige Programmierung

Im vorangehenden Kapitel haben wir kontinuierliche lineare Optimierungsprobleme betrachtet. Wir werden jetzt auf den Fall eingehen wenn Nebenbedingungen sowie zulässige Lösungen aus den ganzen Zahlen sind. Unser Optimierungsproblem liest nun wie folgt

$$\begin{aligned} c^T x &\rightarrow \min \\ Ax &\leq b \quad A \in \mathbb{Z}^{M \times n} \\ x &\in \mathbb{Z}_+^n \end{aligned}$$

Wir haben ja bereits in der Einleitung gesehen, daß viele relevante Probleme in der Praxis eben genau ganzzahlige Programme sind. Die ganzzahlige Nebenbedingung erschweren die Theorie sowie die genau Berechnung von Lösungen. Es gibt allerdings Verfahren welche zBsp. ganzzahlige Programme auf lineare zurückführen kann. Dieses lineare Programme berechnen dann die exakte zulässige Lösung oder berechnen eine approximative Lösung. Mit approximativen Lösungen beschäftigen wir uns dann im nächsten Kapitel. Hier werden wir zunächst Begriffe und einige Ergebnisse zu ganzzahligen Polyeder einführen und zeigen unter welchen Bedingungen das ganzzahlige Programm in ein äquivalentes lineares Programm ausdrücken lässt.

4.1 Polyeder und ganzzahlige Polyeder

Definition 4.1.1 (Polyeder).

Sei $P \in \mathbb{R}^{m \times n}$. Die Menge P ist ein Polyeder, wenn ein $A \in \mathbb{R}^{m \times n}$ und $b \in \mathbb{R}^m$ existieren, sodaß folgendes gilt

$$P = \{x \in \mathbb{R}^n \mid Ax \leq b\}.$$

Wenn P zusätzlich beschränkt ist, dann ist P ein Polytop.

Wenn A, b gegeben, dann ist ein Polyeder definiert

$$\mathcal{P}(A, b) := \{x \in \mathbb{R}^n | Ax \leq b\}.$$

Mit dieser Definition können wir (LP) aus dem vorherigen Kapitel auch wir folgt anschreiben

$$c^T x \rightarrow \min \quad \text{NB } x \in \mathcal{P}(A, b).$$

Allerdings ist das zum Rechnen am Computer nicht sehr sinnvoll, da die Menge nur implizit angegeben ist.

Definition 4.1.2 (Ganzzahlige Polyeder).

$$\mathcal{P}(A, b) := \{x \in \mathbb{R}^n | Ax \leq b\}.$$

4.2 Unimodulare und total unimodulare Matrizen

4.2.1 Unimodulare Matrizen und Transformationen

4.2.2 Hermite Normalform

4.3 Formulierung als ganzzahlige programme